

Documentação TP1

Rafael Santos Oliveira

1 Introdução

A navegação autônoma de robôs móveis é uma área essencial da robótica, com aplicações que vão desde a logística industrial até a exploração de ambientes complexos e desconhecidos. Um dos desafios nesse campo é o *planejamento de caminho* (*path planning*), que consiste em encontrar uma trajetória livre de colisões entre uma posição inicial e um destino final em um ambiente com obstáculos [1].

O objetivo deste trabalho é implementar três algoritmos clássicos de planejamento de caminho: o método baseado em grafos *Roadmap*, o método reativo de *Campos Potenciais* e o *Rapidly-exploring Random Tree* (RRT). Foram considerados dois cenários distintos: um mapa simples, com poucos obstáculos, e um mapa mais complexo, contendo maior densidade de obstáculos. Além disso, foram utilizados dois modelos de robôs diferentes: o robô holonômico Robotino (para os métodos *Roadmap* e RRT) e o robô diferencial Pioneer 3-DX (para o método de *Campos Potenciais*).

1.1 Roadmap

O método *Roadmap* utilizado neste trabalho analisa o mapa, que é uma imagem, criado por meio de um editor de imagens, onde as regiões pretas representam os obstáculos e as áreas claras indicam o espaço livre para navegação. A partir dessa imagem, é gerado um grid com tamanho especificado, neste caso, de 1×1 metro, em que o ponto central de cada grid se torna um nó do grafo. As regiões correspondentes a obstáculos são desconsideradas, e aplica-se um algoritmo de busca em grafos, utilizando a função `nx.shortest_path` da biblioteca `NetworkX` do Python, para determinar uma rota válida entre o ponto inicial e o destino.

1.2 Campos Potenciais

O método de *Campos Potenciais* modela o robô como uma partícula que se move em um campo de forças artificiais. O objetivo (destino) gera uma força atrativa, que puxa o robô em sua direção, enquanto os obstáculos produzem forças repulsivas, que o afastam das regiões perigosas. O caminho é obtido ao seguir o gradiente descendente do potencial total resultante. A principal vantagem deste método é sua simplicidade e natureza reativa; entretanto, ele é suscetível à ocorrência de mínimos locais, que podem impedir o robô de alcançar o destino.

1.3 Rapidly-exploring Random Tree (RRT)

O *Rapidly-exploring Random Tree* (RRT) é um algoritmo de planejamento de caminho baseado em amostragem, que explora o espaço de configurações de forma incremental e eficiente. A partir de um ponto inicial, o algoritmo constrói uma árvore de busca expandindo-a em direção a amostras aleatórias no espaço, buscando conectar o nó mais próximo à nova amostra, respeitando as restrições impostas pelos obstáculos do ambiente.

2 Implementação

Para o desenvolvimento deste trabalho, foram criados dois mapas distintos, um considerado de maior dificuldade e outro de menor complexidade. Ambos foram elaborados utilizando o software *GIMP*. Para o mapa que foi processado pelos algoritmos, foi aplicado um aumento no tamanho dos obstáculos, de modo a considerar o diâmetro do robô durante o processo de planejamento de trajetória, garantindo assim que o robô não ficasse preso em obstáculos.

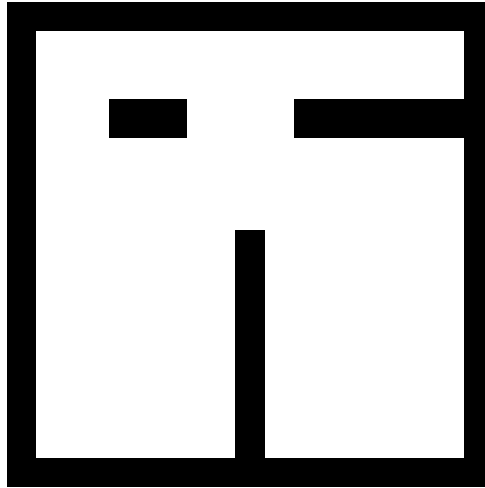


Figura 1: Mapa 1 — cenário de menor complexidade.

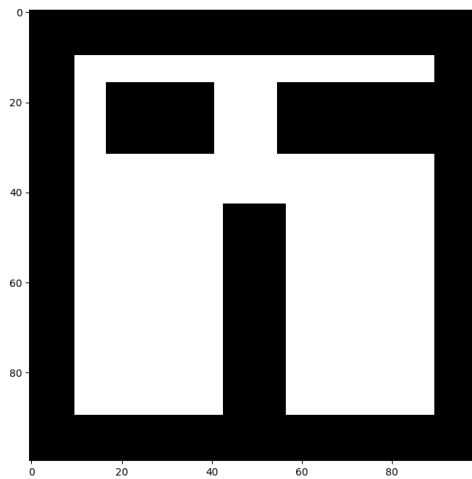


Figura 2: Mapa 1 — cenário de menor complexidade com obstáculos inflados.

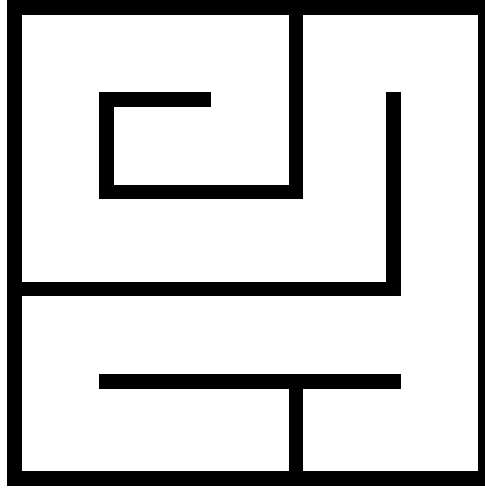


Figura 3: Mapa 2, cenário de maior complexidade.

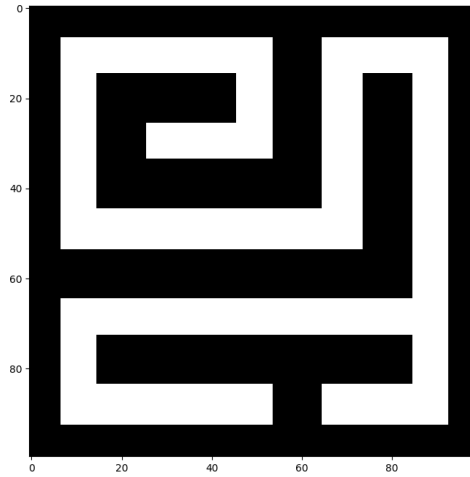


Figura 4: Mapa 2, cenário de maior complexidade, com obstáculos inflados.

2.1 Roadmap

O código responsável pela geração do *Roadmap* é estruturado em torno da função `mapa()`. Essa função recebe como parâmetros o caminho da imagem do mapa, os nós de início e fim, a escala da imagem e o tamanho das células. A partir desses dados, são extraídas as informações referentes aos obstáculos, criado o grid do ambiente e definidos os vértices do grafo, sendo cada célula de 1×1 metro representada por um quadrado da imagem, cujo vértice é o centro dessa região.

Em seguida, os nós que correspondem a áreas de obstáculos são removidos do grafo, resultando em um conjunto de vértices navegáveis. A partir dos nós de início e destino (`start_node` e `end_node`), é realizada a busca do caminho viável dentro do ambiente.

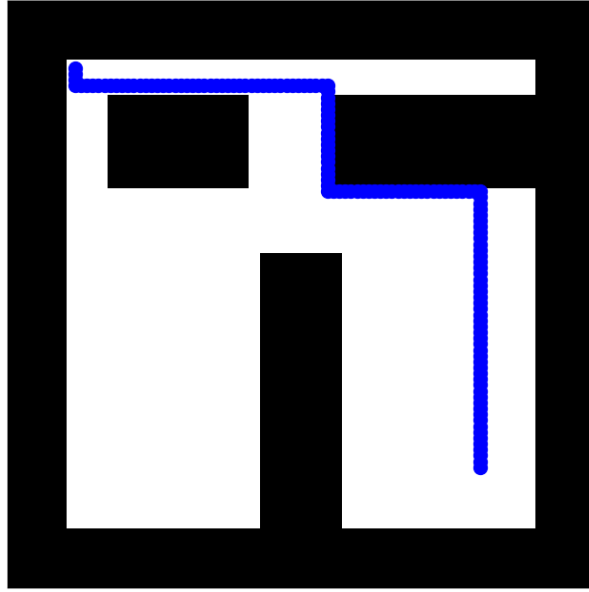


Figura 5: Caminho obtido através da busca em grafos no mapa 1.

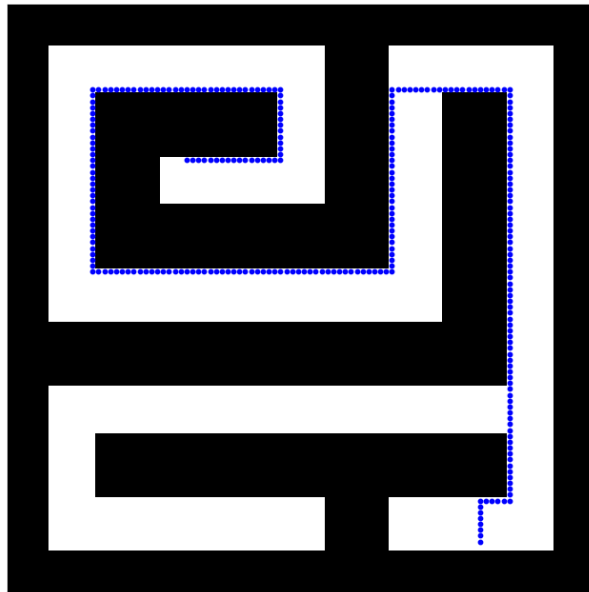


Figura 6: Caminho obtido através da busca em grafos no mapa 2.

Com o grid definido, as informações de posição foram transmitidas ao robô no ambiente de simulação. Para isso, foi implementada a função `pixel_to_world()`, responsável por converter as coordenadas de pixels do mapa em coordenadas do mundo no CoppeliaSim. Essa conversão gera uma lista de nós correspondentes aos pontos que o robô deve percorrer, levando em consideração o tamanho da imagem do grid (neste trabalho, 100 pixels) e a escala física do mapa (10 metros nos cenários utilizados).

Por fim, foi empregada a cinemática inversa de um robô holonômico, representada pela matriz Minv (eq. 1), para o cálculo das velocidades individuais das rodas. Esse procedimento garante que o robô consiga se deslocar de forma suave e precisa até cada posição desejada, seguindo o caminho previamente planejado.

$$M_{dir} = \begin{bmatrix} -\frac{r}{\sqrt{3}} & 0 & \frac{r}{\sqrt{3}} \\ \frac{r}{3} & -\frac{2r}{3} & \frac{r}{3} \\ \frac{r}{3L} & \frac{r}{3L} & \frac{r}{3L} \end{bmatrix} \quad (1)$$

Quando o robô atinge o goal, o *loop* de controle que o direcionava para as posições é finalizado. Em seguida, as velocidades das rodas são ajustadas para zero e a execução do código é encerrada.

2.2 Campos Potenciais

Para esta etapa, foi utilizado como base o código desenvolvido no TP1, empregando o robô *Pioneer P3-DX*. Nesse código, o sensor utilizado é o *Hokuyo LIDAR*, responsável pela detecção de obstáculos no ambiente. A classe `HokuyoSensorSim()` é responsável por obter as leituras do sensor, enquanto a função `transform_laser_to_global()` realiza a transformação das medições do referencial do sensor para o referencial global do robô.

O controle de navegação adotado baseia-se no método proposto por *Desai et al. (1998)*, implementado para guiar o robô até o (*goal*) evitando colisões. A posição do objetivo é convertida do espaço de pixels para o espaço do simulador através da função `pixel_to_world()`. As forças que regem o movimento do robô são calculadas pelas funções `att_force()` e `rep_force()`, que constituem o núcleo do controle de campos potenciais.

A força atrativa (`att_force()`) é direcionada diretamente ao objetivo, enquanto a força repulsiva (`rep_force()`) atua sobre os obstáculos próximos. Esta última avalia a distância entre o robô e cada obstáculo e, se estiver dentro de um raio de influência R , aplica uma força repulsiva proporcional à proximidade — sendo mais intensa quanto mais próximo o obstáculo estiver. Além disso, é adicionado um pequeno componente tangencial para auxiliar o desvio lateral, evitando que o robô apenas recue. Por fim, a força resultante é limitada por um valor máximo (`max_f_rep`).

Adicionalmente, a função `filter_valid_points` limita a quantidade de pontos detectados pelo sensor, evitando leituras redundantes e reduzindo o ruído das medições. Já a função `escape_if_stuck` atua quando o robô fica preso em um obstáculo por um período prolongado, aplicando uma velocidade linear de recuo e uma componente angular aleatória, com o intuito de liberar o robô da situação de bloqueio.

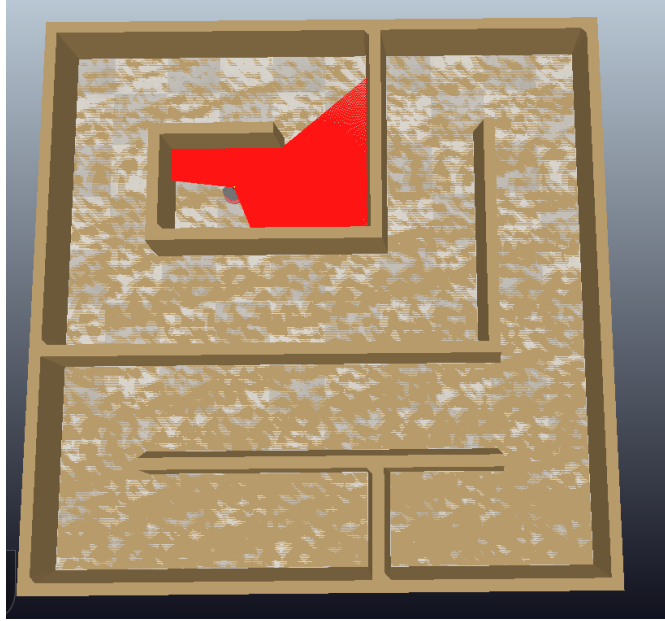


Figura 7: Robô com o sensor a laser ativado durante a execução do campo potencial.

2.3 RRT

O método *Rapidly-Exploring Random Tree* (RRT) segue uma estrutura semelhante à utilizada no código do *RoadMap*. Inicialmente, realiza-se a extração do *grid* de obstáculos a partir da imagem do mapa. Em seguida, o algoritmo RRT é implementado para explorar o ambiente de forma incremental, buscando conectar o ponto inicial ao *goal* por meio da expansão aleatória de uma árvore de busca.

Cada nó da árvore é representado pela classe `Node()`, que armazena suas coordenadas e a referência para o nó pai, permitindo reconstruir o caminho ao final da execução. A função `dist()` calcula a distância euclidiana entre dois pontos, enquanto `sample_free()` gera amostras aleatórias no espaço de busca, com uma pequena probabilidade de selecionar diretamente o objetivo — o que acelera a convergência do algoritmo. A função `nearest()` identifica o nó existente mais próximo da amostra gerada, e `steer()` cria um novo nó na direção dessa amostra, limitado por um tamanho de passo máximo (`step_size`).

A função `collision_free()` verifica se o segmento entre dois pontos está livre de colisões, realizando uma amostragem intermediária ao longo da trajetória proposta. Caso o novo nó seja válido e livre de obstáculos, ele é adicionado à árvore. Quando o novo nó encontra-se suficientemente próximo do objetivo e o caminho até ele também é livre de colisão, considera-se que o objetivo foi alcançado.

Após a construção da árvore, a função `reconstruct_path()` é utilizada para reconstruir o trajeto do ponto inicial até o destino, seguindo as referências entre os nós pais. Em seguida, a função `shortcut()` aplica um pós-processamento de suavização no caminho, eliminando pontos intermediários desnecessários e tornando o trajeto mais eficiente.

A função principal `rrt_planner()` coordena todas essas etapas, gerenciando a expansão da árvore, a verificação de colisões e a visualização gráfica do mapa e da trajetória resultante.

Após a obtenção do caminho, foi realizada a conversão das coordenadas de pixels para o espaço do simulador utilizando a função `pixel_to_world`. O mesmo controlador cinemático empregado no *RoadMap* é então aplicado, permitindo que o robô siga o

caminho planejado de forma suave e contínua.

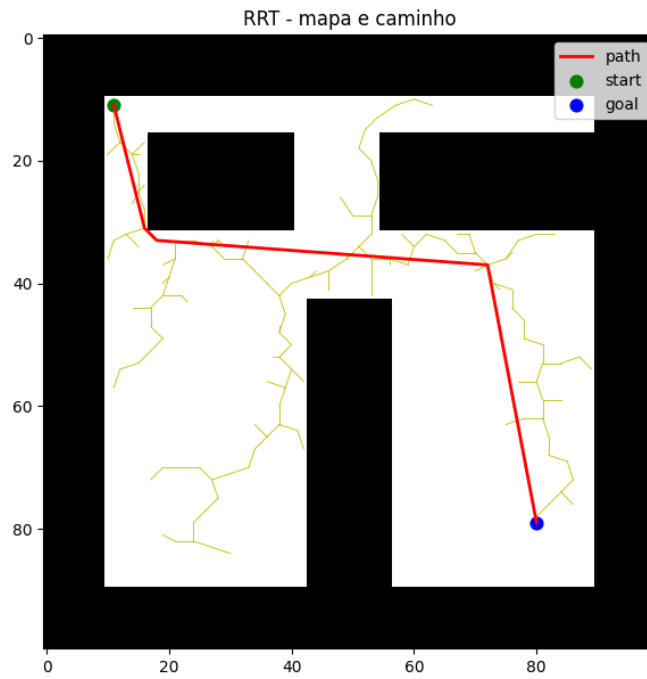


Figura 8: Caminho mapa 1, calculado pelo RRT.

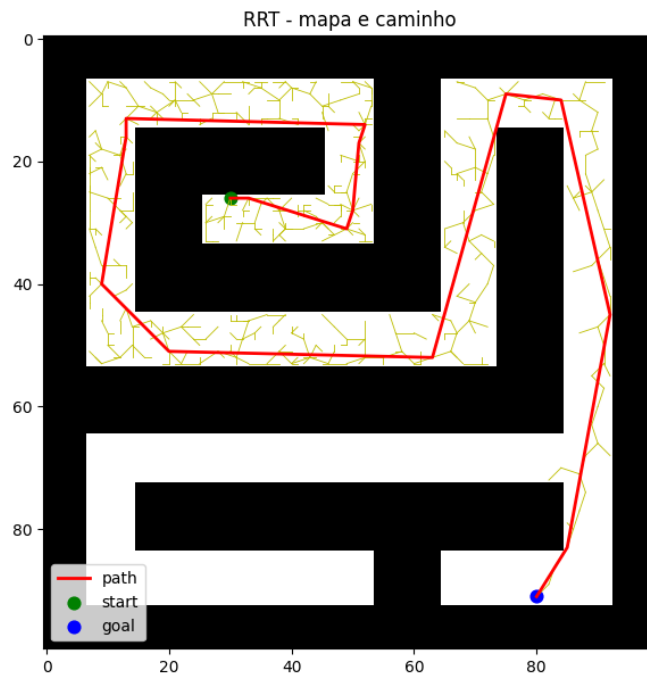


Figura 9: Caminho mapa 2, calculado pelo RRT.

3 Testes

Para os testes, foram selecionados dez pontos em cada mapa de forma aleatória, organizados em cinco pares, sendo que alguns foram ajustados de maneira arbitrária para

garantir uma melhor distribuição e qualidade dos experimentos. Em cada par, o primeiro ponto corresponde à posição inicial do robô, enquanto o segundo representa o `goal`. Cada conjunto de pontos foi utilizado como entrada nos diferentes códigos implementados, registrando-se tanto o tempo necessário para que o algoritmo encontrasse o caminho até o objetivo quanto o tempo total que o robô levou para efetivamente alcançar o destino. Os resultados obtidos foram reunidos em tabelas, permitindo a comparação de desempenho entre os métodos analisados.

Inicial (pixel)	Goal (pixel)	Tempo encontrar goal (s)	Tempo chegar goal (s)
(79,80)	(13,29)	0.5	29.7
(42,20)	(13,25)	0.6	9.7
(14,18)	(68,63)	0.5	24.8
(72,87)	(80,57)	0.6	35.3
(81,36)	(89,79)	0.6	33.4

Tabela 1: Tabela com os resultados obtidos utilizando o algoritmo Roadmap no Mapa 1.

Inicial (pixel)	Goal (pixel)	Tempo encontrar goal (s)	Tempo chegar goal (s)
(79,80)	(13,29)	0.9	9.1
(42,20)	(13,25)	0.6	4.7
(14,18)	(68,63)	0.8	8.9
(72,87)	(12,87)	0.8	13.1
(81,36)	(89,79)	0.5	12.8

Tabela 2: Tabela com os resultados obtidos utilizando o algoritmo RRT no Mapa 1.

Inicial (pixel)	Goal (pixel)	Tempo para chegar no goal (segundos)
(79,80)	(13,29)	1m 10s
(42,20)	(13,25)	1m 7s
(14,18)	(68,63)	1m 15s
(72,87)	(12,87)	não conseguiu
(81,36)	(89,79)	não conseguiu

Tabela 3: Tabela com os resultados obtidos utilizando o algoritmo de campos potenciais no Mapa 1.

Inicial (pixel)	Goal (pixel)	Tempo encontrar goal (s)	Tempo chegar goal (s)
(90,89)	(90,45)	0.7	37.3
(28,36)	(10,13)	0.5	9.7
(11,8)	(52,41)	0.5	20
(32,65)	(67,74)	0.6	24.4
(7,14)	(54,89)	0.6	50.8

Tabela 4: Tabela com os resultados obtidos utilizando o algoritmo Roadmap no Mapa 2.

Inicial (pixel)	Goal (pixel)	Tempo encontrar goal (s)	Tempo chegar goal (s)
(90,89)	(90,45)	0.6	18.13
(28,36)	(10,13)	0.4	7
(11,8)	(52,41)	0.8	7.2
(32,65)	(67,74)	0.8	11.2
(7,14)	(54,89)	0.9	19.7

Tabela 5: Tabela com os resultados obtidos utilizando o algoritmo RRT no Mapa 2.

A baixo está os pontos iniciais e finais dos testes utilizados para o mapa 1.

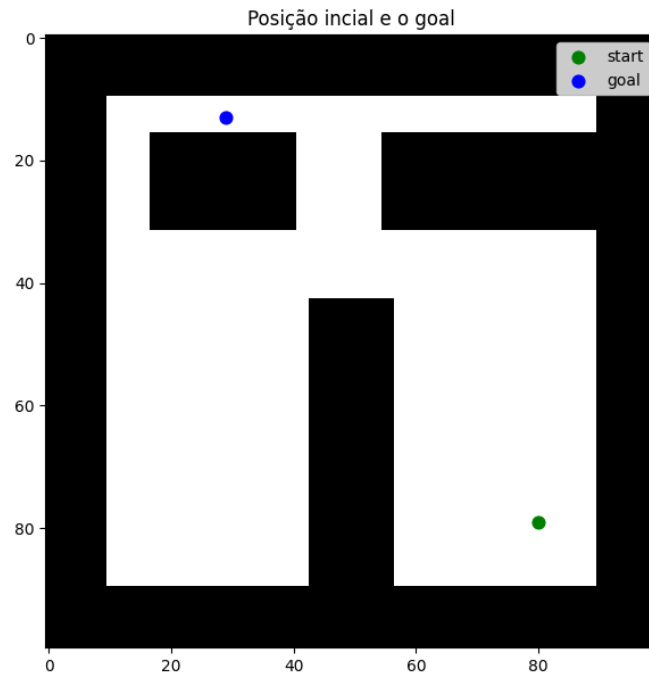


Figura 10: Mapa 1 com ponto iniciais (79,80) e goal (13,29).

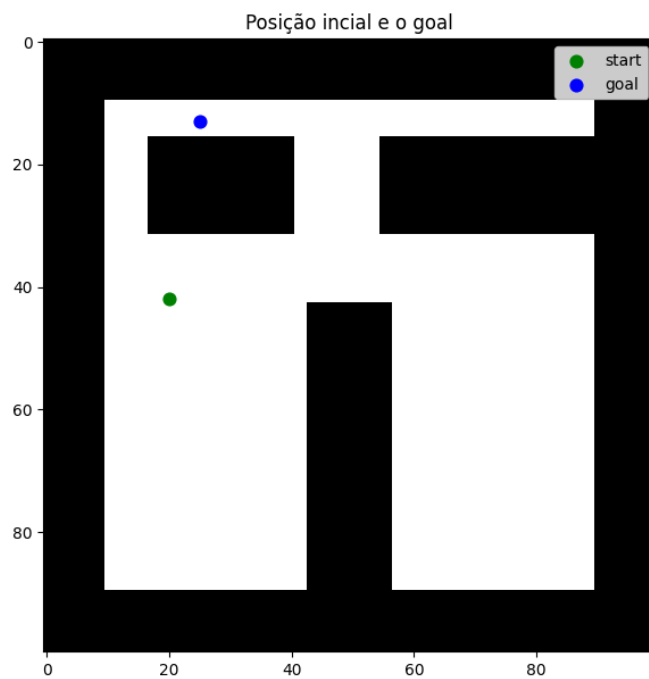


Figura 11: Mapa 1 com ponto inicial (42,20) e goal (13,25).

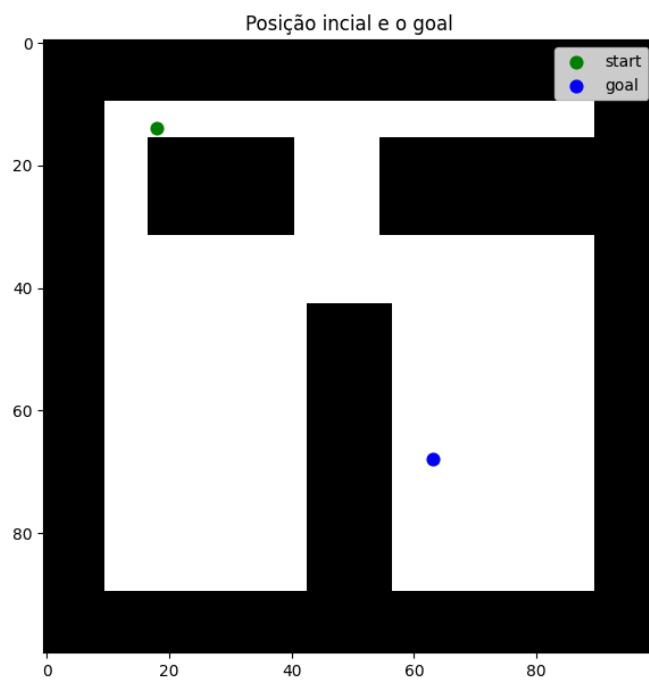


Figura 12: Mapa 1 com ponto inicial (14,18) e goal (68,63).

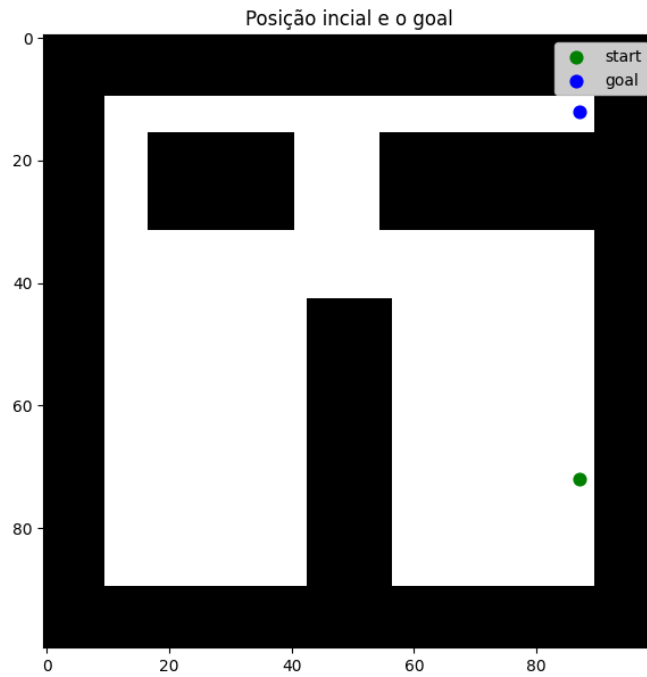


Figura 13: Mapa 1 com ponto inicial (72,87) e goal (12,87).

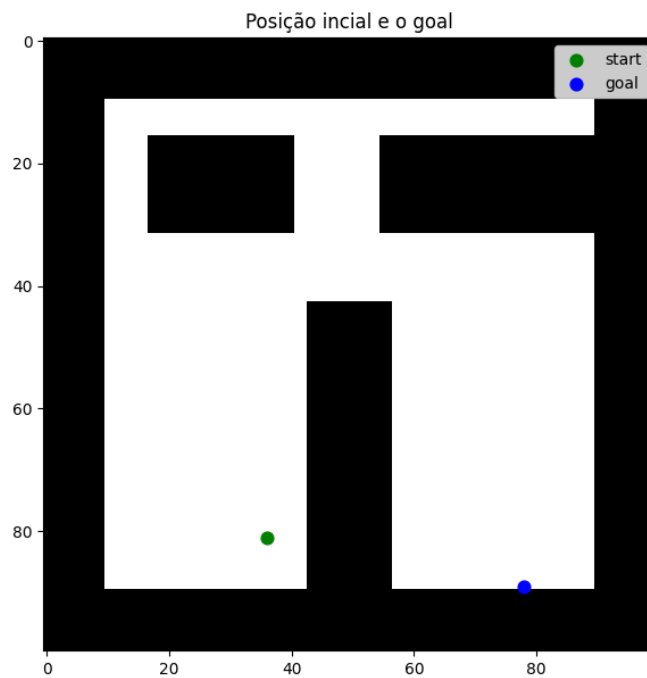


Figura 14: Mapa 1 com ponto inicial (81,36) e goal (89,79).

A baixo está os pontos iniciais e finais dos testes utilizados para o mapa 2.

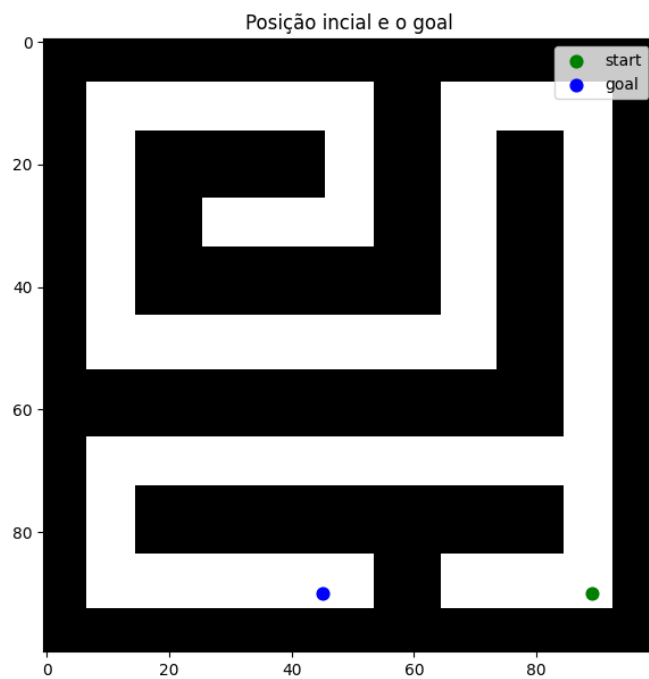


Figura 15: Mapa 1 com ponto iniciais (90,89) e goal (90,45).

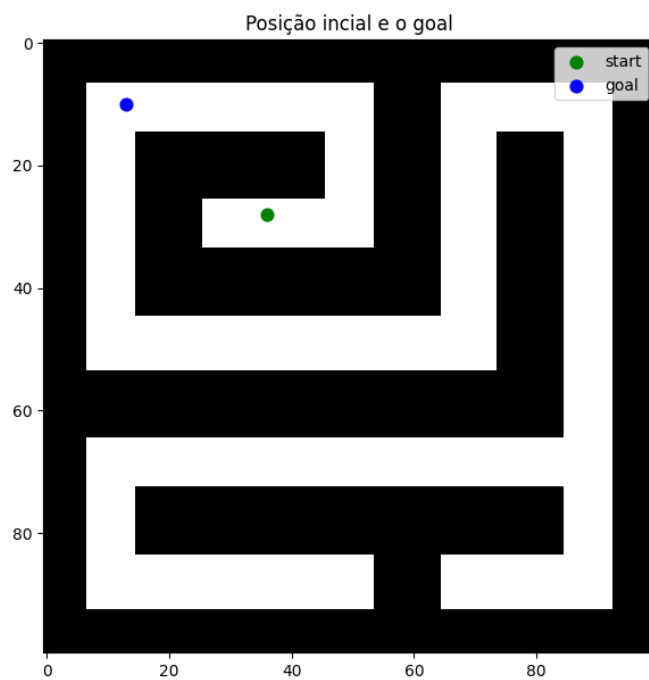


Figura 16: Mapa 1 com ponto iniciais (28,36) e goal (10,13).

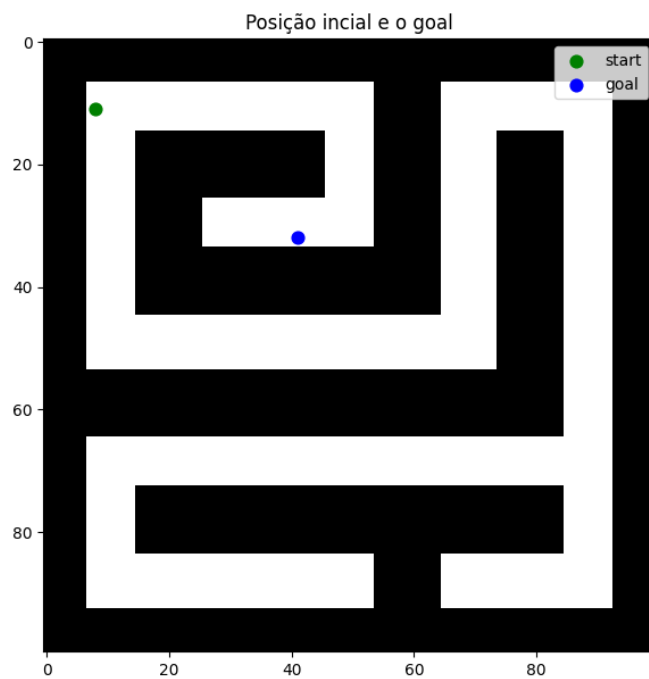


Figura 17: Mapa 1 com ponto iniciais (11,8) e goal (52,41).

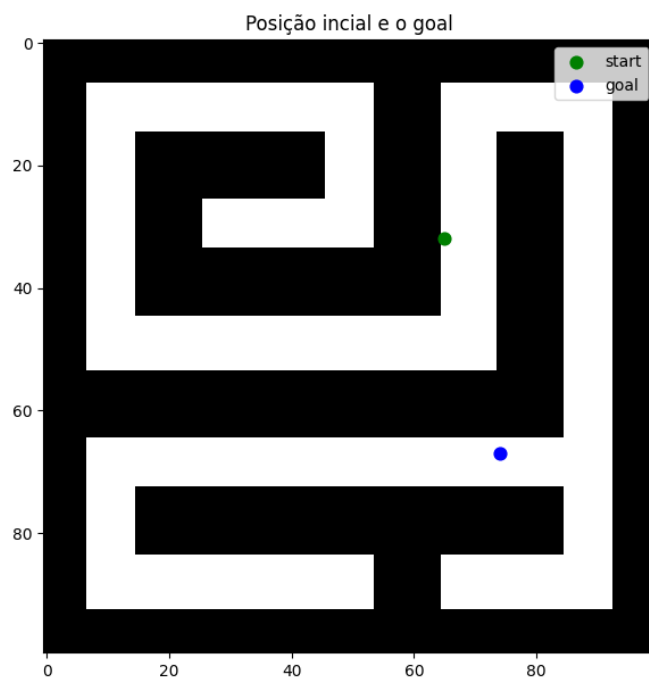


Figura 18: Mapa 1 com ponto iniciais (32,65) e goal (67,74).

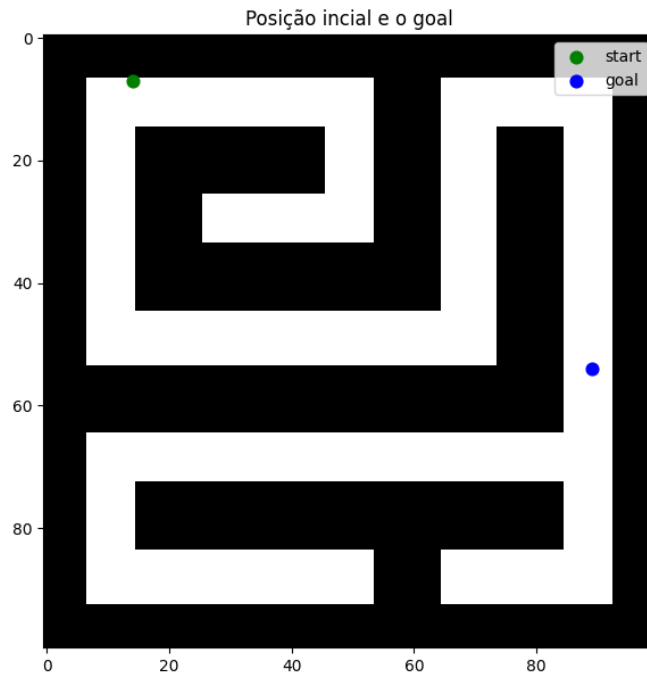


Figura 19: Mapa 1 com ponto iniciais (7,14) e goal (54,89).

4 Conclusão

A implementação dos algoritmos deliberativos apresentou resultados bastante satisfatórios. Todos os métodos foram capazes de encontrar o caminho até o objetivo, inclusive nos mapas mais complexos. Durante os testes, observou-se que, embora o algoritmo RRT apresente certa variação no tempo necessário para encontrar a rota, sendo em alguns casos ligeiramente mais lento, o fato de gerar trajetórias com menos pontos faz com que o robô alcance o destino mais rapidamente.

Em contrapartida, o método reativo baseado em Campos Potenciais apresentou maiores dificuldades para atingir o objetivo. O tempo de convergência foi significativamente mais alto, e ocorreram problemas na aplicação correta da força de repulsão. Além disso, foi observado que, quando o obstáculo possui dimensões muito grandes, o vetor atrativo continua puxando o robô na direção do alvo, enquanto a força repulsiva não é suficiente para afastá-lo completamente. Uma possível melhoria seria aumentar o número de vetores de repulsão detectados pelo sensor, o que poderia aprimorar o comportamento de desvio. Entretanto, não consegui implementar essa modificação a tempo. Esse fenômeno foi especialmente evidente no segundo mapa, que apresenta um layout semelhante a um labirinto, caracterizado por obstáculos contínuos.

Um exemplo da execução dos testes pode ser visualizado no seguinte link:
https://youtu.be/hc0D0_4zISc

Outro desafio enfrentado foi a implementação de mapas mais complexos. Inicialmente, ocorreram erros na conversão entre as coordenadas em pixels e o espaço do CoppeliaSim, o que fazia o robô colidir ou permanecer preso. Esse problema foi solucionado ao aumentar a espessura dos obstáculos durante a geração dos mapas e ao utilizar dimensões fixas de 100 pixels e escala de 10 metros no CoppeliaSim, garantindo assim o funcionamento correto da função de conversão.

Referências

- [1] Roland Siegwart, Illah Reza Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. Intelligent Robotics and Autonomous Agents Series. The MIT Press, Cambridge, MA, 2nd edition, 2011.